Customer Churn Predictor

End-to-End Customer Churn Prediction System

Tools: Python (scikit-learn), FastAPI, Streamlit, Evidently, SHAP, Tableau

Mitra Boga (*2023005779*)
Akshith Varma Datla (*2023006014*)
Sai Yashweer Potelu (*2023006041*)
Pranav Surya (*2023003386*)

INTN2333
Dr. Giddaluru Soma Sekhar, Ph.D
March 20th, 2026

Abstract

This project builds a complete churn prediction system to identify customers likely to discontinue a subscription-based service. Rather than stopping at model accuracy, the solution converts churn probabilities into a **business decision policy** ("intervene or not") using a **profit-based threshold selection strategy**. The pipeline includes data ingestion, cleaning, feature preprocessing, classification modelling, evaluation with appropriate metrics for imbalanced data, explainability using both global and local feature attribution methods, and a business simulation to quantify expected value across thresholds. The final system is deployed as a FastAPI backend with logging, a modern Streamlit dashboard interface, drift monitoring using Evidently, and a Tableau dashboard suite built from exported analytics datasets.

---

1. Introduction and Problem Definition

Customer churn is a major financial risk in subscription and service industries because customer acquisition costs are typically high, and losing customers reduces recurring revenue. The goal of this case study is to build a system that:

1. Predicts **churn probability** for each customer using historical attributes.
2. Explains the **drivers** behind churn risk.
3. Converts predictions into an **actionable strategy** (who to target).
4. Quantifies the strategy using an **ROI simulation** (profit-aware threshold selection).
5. Packages results in a stakeholder-friendly manner using **Tableau dashboards** and a usable web UI.

---

2. Project Objectives

2.1 Technical objectives

- Build a reproducible ML pipeline using scikit-learn preprocessing and modeling tools.
- Compare multiple classification models and select the best based on ranking metrics.

- Evaluate performance using metrics suited for churn (imbalanced classification).
- Generate explainability outputs (global and local).
- Deploy the model via API and a UI for interactive inference.
- Add monitoring to detect input distribution drift over time.

2.2 Business objectives

- Use churn probabilities to support retention decisions.
- Determine an intervention threshold based on expected profit rather than arbitrary defaults.
- Provide dashboards that help non-technical users explore churn patterns and decisions.

---

3. Dataset and Features

3.1 Dataset description

The project uses the **Telco Customer Churn** dataset (a standard churn dataset containing customer demographics, subscription characteristics, billing, and churn label).

3.2 Target variable

- **Churn** (binary):
  - 1 = customer churned
  - 0 = customer stayed

3.3 Key features used

Examples include:

- Tenure, MonthlyCharges, TotalCharges
- Contract type (month-to-month / one year / two years)
- Internet Service type
- PaymentMethod
- Support and add-on services (TechSupport, OnlineSecurity, etc.)

- Demographic fields (gender, SeniorCitizen, Partner, Dependents)

3.4 Cleaning and preprocessing

Key cleaning steps:

- Converted `TotalCharges` to numeric (handling missing/blank values).
- Standardized target `Churn` into 0/1.
- Used an end-to-end pipeline for reproducible preprocessing:
    - Numeric: median imputation + standard scaling
    - Categorical: mode imputation + one-hot encoding

This ensures the same transformations are applied at both training time and inference time.

---

4. Methodology and Workflow

4.1 Overall Approach

The project was developed using a structured, industry-aligned workflow that mirrors real applied machine learning projects. Instead of treating this as a one-time classification exercise, we built a complete pipeline that supports reproducibility, interpretability, decision-making, deployment, and monitoring. This ensures that the outcome is not only technically correct, but also operationally realistic and stakeholder-ready.

The workflow was designed to produce **clear artifacts at each stage** (clean datasets, trained models, metrics files, plots, Tableau exports, and monitoring reports), allowing the project to be validated and presented reliably.

4.2 Step-by-Step Workflow

Step 1: Problem Definition

We defined the problem as a binary classification task:

- **Predict whether a customer will churn** based on customer demographic, service usage, billing, and contract data.

However, a key design decision was to treat churn prediction as a *decision-support system*, not merely a predictive system. Therefore, we formalized two outputs:

1. **Churn probability** (continuous risk score)
2. **Retention action decision** (intervene/do not intervene) based on a threshold selected through business simulation.

This distinction ensured that the project focuses on producing actionable outcomes rather than purely maximizing accuracy.

Step 2: Data Collection and Cleaning

The data ingestion module automatically downloads the dataset and produces a cleaned modelling-ready CSV. Cleaning focused on correctness and repeatability:

- Converted `TotalCharges` to numeric (handling blanks and coercing invalid values to missing).
- Standardized the churn label into a binary format (0/1).
- Removed `customerID` for modelling to prevent ID-based leakage and ensure generalizable learning.
- Validated expected schema (ensuring key features exist to prevent silent failures).

The output of this stage is:

- `data/processed/telco_processed.csv`

This stage reflects a core machine learning engineering principle: **training data must be reproducible and verifiable**.

Step 3: Feature Preprocessing Pipeline

Rather than manually preprocessing data outside the model, we implemented a single end-to-end preprocessing pipeline using scikit-learn's `Pipeline` and `ColumnTransformer`. This is important because real deployments require the *exact same preprocessing* at training time and inference time.

- Numeric features: median imputation + standard scaling
- Categorical features: mode imputation + one-hot encoding
- Output: a fully numeric matrix usable by ML models

This design ensures:

- No train-test preprocessing leakage
- Consistency across training, evaluation, and production API inference
- Portability and reusability

Step 4: Modelling and Training

Multiple candidate models were trained under identical preprocessing conditions and compared using consistent evaluation logic. This supports fair model selection.

Key training design choices:

- Train-test split with stratification (preserves churn class ratio in both sets)
- Churn class imbalance handling using class-weighting
- Model selection based on ranking metrics (ROC-AUC, PR-AUC)

The training stage produces:

- Saved trained model artifact (`models/model.joblib`)
- Metadata (`models/metadata.json`)
- Model comparison results (`reports/metrics/model_metrics.csv`)

Step 5: Evaluation and Threshold Analysis

Evaluation was designed for **imbalanced classification**, where accuracy can be misleading.

We used:

- ROC-AUC (ranking quality)
- PR-AUC (better for positive-class performance)
- confusion matrix
- precision/recall and threshold scanning

A threshold scan was included because churn systems are often deployed with thresholds that reflect business capacity (how many customers can be targeted) and the cost of false positives.

This stage produces:

- `reports/metrics/threshold_scan.csv`

Step 6: Explainability and Transparency

To make the model trustworthy and presentation-ready, we added explainability mechanisms that answer:

- **Which features drive churn risk overall?**
- **Why was a specific customer flagged?**

Methods included:

- Permutation importance (global)
- SHAP global importance (global)
- SHAP local example (per-customer explanation)

This stage produces:

- `reports/figures/permutation_importance_top15.png`
- `reports/figures/shap_global_importance_top15.png`
- `reports/figures/shap_local_waterfall_example.png`

Step 7: Business Simulation and Decision Policy

A major project differentiator is that it defines a **profit-optimized retention decision policy** rather than using a default 0.5 threshold.

We simulated the expected value based on:

- offer cost
- save rate
- profit margin
- months saved
- churn probability

This produces:

- `reports/figures/business_total_value_vs_threshold.png`
- `reports/metrics/best_threshold.json`

The end result is a recommendation that explicitly answers:

> "At what probability should we intervene to maximize business value?"

Step 8: Deployment and User Interface

The project includes:

- **FastAPI backend** for single and batch predictions
- **Streamlit UI** as a modern SaaS dashboard for stakeholders

This converts the system from a notebook-style demo into a usable application.

The API logs all predictions, enabling monitoring and auditing.

Step 9: Monitoring and Model Lifecycle Support

To handle real-world data drift, we built monitoring using Evidently:

- compares reference sample (saved during training) vs current prediction logs
- generates a drift report HTML

Output:

- `reports/monitoring/data_drift_report.html`

This supports the lifecycle requirement that models should be validated after deployment.

Step 10: Tableau Visualization Suite

Finally, Tableau is used as the primary business visualization platform.

We export Tableau-ready datasets:

- cleaned dataset
- scored dataset
- ROI threshold table
- feature importance table

This supports professional stakeholder communication and aligns with modern analytics workflows where ML outputs are commonly consumed through dashboards.

4.3 Summary of Workflow Deliverables

The workflow produces a clear “artifact trail”:

- **Cleaned data:** `telco_processed.csv`
- **Model artifact:** `model.joblib`
- **Evaluation metrics:** `model_metrics.csv`, `threshold_scan.csv`
- **Explainability plots:** permutation + SHAP images
- **Business decision:** `best_threshold.json`, ROI plots

- **Deployment:** FastAPI + Streamlit
- **Monitoring:** drift report
- **Tableau exports:** clean/scored/ROI/importance CSVs

This structure makes the project reproducible, testable, and easy to present.

---

5. Machine Learning Modelling

5.1 Why Customer Churn is a Classification Problem

Customer churn prediction is naturally modelled as a **binary classification task**:

- Class 1: churn (customer leaves)
- Class 0: no churn (customer stays)

Unlike regression problems, the objective is not predicting a continuous outcome (e.g., price or sales), but rather learning the probability of a churn event occurring, which can later be converted into a decision rule.

5.2 Model Development Strategy

The project uses a standard modelling strategy used in professional ML settings:

1. Start with a strong, interpretable baseline model
2. Add a nonlinear model to capture interaction effects
3. Compare models using robust metrics
4. Select the model that balances performance + interpretability + deployment simplicity

This avoids overengineering while still ensuring strong predictive performance.

5.3 Models Implemented

5.3.1 Logistic Regression (Baseline + Final Model)

Logistic Regression was implemented as the primary baseline and ultimately selected as the final deployed model.

Reasons it is appropriate for churn:

- Strong performance on tabular, structured data
- Produces calibrated probabilities reasonably well
- Coefficients can be interpreted alongside feature importances
- Easy to deploy and fast at inference time

We also used:

- **class_weight="balanced"** to address churn class imbalance (reducing bias toward the majority class)

5.3.2 Random Forest (Nonlinear Candidate Model)

Random Forest was implemented as a comparison model because:

- It captures nonlinear relationships and interactions without heavy feature engineering.
- It is robust to noise and complex categorical interactions after encoding.

We also used:

- **class_weight="balanced_subsample"** to reduce imbalance bias.

Random Forest is often a strong second candidate in churn projects, even if not ultimately selected.

5.4 Model Selection and Results

Models were evaluated on a held-out test split using ROC-AUC and PR-AUC.

Final results:

- **Logistic Regression**
  - ROC-AUC: 0.8414
  - PR-AUC: 0.6324
- **Random Forest**
  - ROC-AUC: 0.8224
  - PR-AUC: 0.6135

Logistic Regression outperformed Random Forest across both ranking metrics, so it was selected as the final model.

5.5 Why ROC-AUC and PR-AUC Were Used

Customer churn is typically imbalanced (fewer churners than non-churners). In such cases:

- **Accuracy can be misleading** because a model could predict “no churn” for everyone and still appear strong.
- ROC-AUC measures how well the model ranks churners above non-churners.
- PR-AUC focuses more directly on positive-class performance (churners), making it a strong metric when churn is the class of interest.

This evaluation approach is widely accepted in industry for churn prediction and other imbalanced classification tasks.

5.6 Thresholding and Decision Support

The model output is a **probability**. The business must convert this into a decision:

- Target retention offers to customers above a threshold.

A key project feature is that we do not use threshold=0.5 arbitrarily. Instead, we evaluate threshold tradeoffs:

- Precision and recall change across thresholds.
- A threshold scan quantifies these tradeoffs.

- An ROI simulation identifies the threshold that maximizes expected profit.

Thus, the model is deployed not just as a classifier, but as part of a **decision-making pipeline**.

5.7 Explainability as Part of Modelling

Modern churn systems require transparency. Therefore, explainability was not treated as an optional add-on, but as part of the modelling process:

- Permutation importance provides global driver interpretation.
- SHAP provides both global and local contribution explanations.

This supports trust and actionable business insight rather than “black-box predictions.”

5.8 Summary of Modelling Outcomes

The final modelling component provides:

- A trained churn probability model with strong ranking performance
- Robust evaluation across metrics relevant to churn
- Threshold analysis for operational decision-making
- Explainability tools for stakeholder communication

In combination with the ROI simulation, deployment, monitoring, and dashboards, this modelling approach is aligned with real-world churn analytics systems.

---

6. Model Evaluation and Interpretation

6.1 Confusion matrix at threshold 0.50

At a default classification threshold of 0.50:

```
[[746 289]
 [ 81 293]]
```

This reflects the standard churn tradeoff: increasing recall typically reduces precision. Therefore, selecting an operating threshold must be aligned with business constraints and costs.

6.2 Threshold scan (precision/recall tradeoff)

Threshold scan showed that:

- Lower thresholds produce higher recall but target more customers
- Higher thresholds increase precision but may miss churners

This motivates the ROI-based decision threshold approach used later.

---

7. Explainability

A key requirement for real-world adoption is explaining **why** customers are flagged as high risk.

7.1 Permutation importance (global)

Permutation importance was computed and saved as:

- `reports/figures/permutation_importance_top15.png`
- `reports/figures/permutation_importance.csv`

This identifies which raw features most influence predictive performance in a business-friendly way.

7.2 SHAP explainability (global + local)

After installing SHAP, the project produced:

- `reports/figures/shap_global_importance_top15.png`
- `reports/figures/shap_local_waterfall_example.png`

Global SHAP explains which factors most contribute overall; local SHAP provides a single-customer explanation that is useful for demos, stakeholder trust, and presentations.

---

8. Business Simulation and Decision Policy (ROI)

8.1 Why threshold selection matters

Using threshold=0.50 is arbitrary. In business settings, the correct threshold depends on:

- cost of retention intervention (offers, discounts, support)
- profit margin
- probability of saving churners
- value retained if churn is prevented

8.2 Business assumptions used

The simulation used:

- offer cost per targeted customer
- save rate if targeted
- profit margin
- months of retained value

8.3 Best ROI threshold

The ROI simulation produced:

- **Best threshold:** ~0.15
- **Targeted customers:** ~4992
- **Total expected value:** ~$198,995

This is a major deliverable:

> The system selects interventions based on expected profit, not just model score.

Artifacts saved:

- `reports/figures/business_total_value_vs_threshold.png`
- `reports/figures/business_simulation_thresholds.csv`
- `reports/metrics/best_threshold.json`

---

9. System Implementation and Architecture

9.1 Repository structure

The project is organized as a production-style Python package:

- `churn/` – ML pipeline, training, evaluation, explainability, business simulation, API, monitoring
- `app/` – Streamlit UI
- `reports/` – saved plots and metrics
- `data/` – local generated data (raw/processed/logs)
- `tableau/` – dashboards and screenshots (final deliverables)

9.2 FastAPI backend (inference service)

The API provides:

- `/health` – status check
- `/predict` – single record inference
- `/predict_batch` – batch inference for CSV-style use

The API logs predictions to:

- `data/logs/predictions_log.csv`
  This log becomes the “current data stream” for monitoring.

9.3 Streamlit Modern SaaS Dashboard UI

9.3.1 Purpose of the Streamlit UI

While the FastAPI service provides a clean machine-to-machine interface, the Streamlit UI serves a different and equally important role: it acts as a **human-facing decision dashboard**. In real organizations, business stakeholders (customer success, retention teams, analysts, and product managers) rarely consume raw API outputs. They require an interface that:

- Makes model predictions understandable at a glance
- Converts probabilities into **clear actions** (intervene vs. no action)
- Allows “what-if” analysis on the intervention threshold and business assumptions
- Supports demonstration and validation during presentations
- Enables lightweight batch scoring without needing to write code

The Streamlit app was therefore designed as the “product layer” of the system.

9.3.2 UI Design Principles (Modern SaaS Style)

To avoid the appearance of a basic form-based prototype, the UI follows modern SaaS design principles:

- **KPI cards at the top** to communicate the system’s current status and key metrics instantly
- **Sectioned layout with tabs** so users can switch between prediction, analytics, explainability, ROI, and logs
- **Card-based components** (gradient containers + soft shadows) to create clear hierarchy and readability
- **Minimal clutter** with consistent spacing, aligned typography, and concise labels

- **Stakeholder-centric language**: “Decision Threshold”, “Expected Value”, “Intervene” rather than purely technical terms

This layout increases usability and makes the demo feel like a real tool rather than a classroom script.

9.3.3 Core User Flows Implemented

A) Single-Customer Prediction (Interactive Scoring)

The primary workflow supports evaluating one customer at a time:

1. User fills a structured form with customer attributes (tenure, contract type, internet service, billing method, etc.).
2. On clicking **Predict**, the UI calls the FastAPI endpoint `/predict`.
3. The backend returns `churn_probability` (a value between 0 and 1).
4. The UI displays:
    - **Probability** (as a percentage)
    - **Risk bucket** (Low / Medium / High)
    - **Business decision** (Intervene / Do Not Target) based on the chosen threshold
    - **Expected Value** estimate per customer under the current business assumptions

This flow is designed for customer success teams who handle individual accounts and want quick prioritization.

B) Demo Presets (Presentation-Friendly)

To make presentations smooth and repeatable, the UI includes **preset customer profiles**, such as:

- High-risk profile (short tenure, month-to-month contract, higher charges)
- Low-risk profile (long tenure, long-term contract, stable billing)
- Custom default profile

This avoids manual typing during demos and ensures the team can quickly showcase differences in model outputs and decisions.

C) Decision Threshold Control (What-if Analysis)

A key improvement over basic churn projects is the ability to interactively adjust the **decision threshold**:

- The threshold determines which customers get targeted for retention actions.
- Users can test “what happens if we become more aggressive or more conservative?”
- The UI also displays the **best ROI threshold** saved from the business simulation (in your case ~0.15).

This is important because it reinforces that the system is not merely predictive—it supports decision-making.

D) Business Assumption Controls (Economic Sensitivity)

The UI includes adjustable business parameters, allowing the user to simulate different retention environments:

- Offer cost ($)
- Save rate (probability of saving a churner if targeted)
- Profit margin
- Months saved (retained value horizon)

This allows stakeholders to observe how intervention profitability changes under different assumptions. In real retention programs, these assumptions can vary by campaign or segment, so the feature increases realism and applicability.

9.3.4 Expected Value (EV) Integration

To directly connect model predictions to business outcomes, the UI computes **Expected Value per targeted customer**:

$$EV = P(churn) \times save_rate \times (margin \times months_saved \times MonthlyCharges) - offer_cost$$

This is displayed alongside the churn probability.

- If EV is positive, targeting that customer is expected to be profitable.
- If EV is negative, the UI suggests raising the threshold or reducing cost assumptions.

This EV framing is a major “portfolio differentiator” because it demonstrates that the model is deployed as a decision tool rather than as a metrics-only artifact.

9.3.5 Analytics Dashboard Tab (SaaS Dashboard View)

The Streamlit UI includes a dedicated analytics section that mirrors modern SaaS dashboards.

It uses the **Tableau export dataset** (`telco_scored.csv`) to visualize:

- Churn probability distribution (binned)
- Targeting decision distribution (stacked view)
- Average churn probability by contract type
- Top churn drivers (feature importance)
- Precision vs recall curves across thresholds
- Top high-risk customer preview table

This is valuable for two reasons:

1. It provides an immediate dashboard without needing Tableau during a live demo.
2. It aligns the UI with business stakeholder expectations (charts + tables + segmentation).

9.3.6 Explainability Tab (Trust and Transparency)

The explainability section displays:

- Permutation importance (global feature impact)
- SHAP global importance (feature contributions across population)
- SHAP local explanation (single-customer waterfall example)

From a user perspective, this answers:

- “Which features drive churn risk overall?”
- “Why was this particular customer flagged?”

This transparency is critical in real-world contexts where decisions may affect budgets, customer interactions, and retention campaigns.

9.3.7 Business Tab (ROI Policy Visualization)

The Business section integrates the ROI layer by showing:

- ROI vs threshold plot
- ROI table (threshold → total expected value)
- Download option for ROI exports

This emphasizes that the system includes a formal policy recommendation:

- Choose the threshold that maximizes expected profit
- Target retention offers accordingly

9.3.8 Logs & Batch Scoring (Operational Capability)

To make the system usable beyond one-off demos:

Logging

Each API prediction is logged to:

- `data/logs/predictions_log.csv`

The UI includes a log viewer to show:

- recently scored customers
- timestamps and probability outputs
- download option for auditing or reporting

Batch scoring

The UI supports uploading a CSV file and scoring it via `/predict_batch`.
Outputs include:

- churn probability for each row
- risk bucket / targeting decision

This is operationally important because retention teams often evaluate thousands of customers at once for campaigns.

9.3.9 Technical Architecture Summary (UI ↔ API)

The UI is architected as a frontend that calls the backend via HTTP:

- Streamlit collects structured inputs
- Sends JSON payload to FastAPI
- Receives probability output
- Displays decision and business metrics
- Logs are written by the API, not the UI (clean separation of concerns)

This separation mirrors real deployment patterns:

- **Model logic stays on the server**
- **UI stays lightweight and presentation-focused**

---

10. Monitoring and Reliability

10.1 Why monitoring matters

In real deployments, customer behaviour and product pricing change over time. This creates **data drift**, which can reduce model reliability.

10.2 Drift reporting

The project uses Evidently to compare:

- **reference data** (saved at training time) vs
- **current data** (from prediction logs)

Output:

- `reports/monitoring/data_drift_report.html`

This adds credibility: the system is designed with lifecycle awareness, not just training-time performance.

---

11. Tableau Dashboards (Primary Visualization Layer)

Tableau is used as the main stakeholder-facing data visualization platform. The project exports Tableau-ready datasets:

- `data/tableau/telco_cleaned.csv`
- `data/tableau/telco_scored.csv`
- `data/tableau/roi_thresholds.csv`
- `data/tableau/feature_importance.csv`

Recommended Tableau dashboard suite (4 dashboards)

1. **Executive Overview**

    - churn rate, average tenure, average monthly charges
    - filters by contract type, internet service, and payment method
2. **Customer Segmentation**
    - churn patterns across Contract, PaymentMethod, TenureBucket, etc.
3. **Risk Scoring Dashboard**
    - churn probability distribution
    - risk buckets/deciles
    - top high-risk customer list
4. **ROI Decision Dashboard**
    - total expected value vs threshold
    - highlights the ROI-max threshold (~0.15)
    - supports decision-making and budget planning

This dashboard set makes the project readable for non-technical stakeholders and strengthens the “business impact” narrative.

---

12. Why This Case Study Was Chosen for INTN2333

Customer churn prediction was selected because it provides:

- **clear real-world business value** (retention is tied to revenue)
- a strong supervised learning classification task
- meaningful evaluation beyond accuracy
- a complete “end-to-end” narrative:
  data → model → evaluation → explainability → decision → deployment → monitoring → dashboards

This aligns strongly with both academic learning goals and portfolio quality expectations.

---

13. Limitations

- The ROI simulation uses simplified assumptions (fixed save rate, fixed margin, fixed months saved). Real organizations may use customer lifetime value (CLV) estimates per segment.
- The dataset lacks geographic fields, so map-based dashboards are not applicable.
- Monitoring is a report-based workflow; real production systems may require automated alerts and scheduled retraining triggers.

---

14. Future Enhancements

- Add CLV modelling to replace simplified retained value assumptions.
- Add an automated retraining workflow when the drift exceeds thresholds.
- Add model calibration to improve probability reliability.
- Deploy publicly (Streamlit Community Cloud or cloud platform) for a true production demo.
- Expand feature engineering and add XGBoost as an additional model candidate (optional).

---

15. Conclusion

CustomerChurnPredictor demonstrates a complete churn analytics and prediction system. The project goes beyond training a classifier by integrating explainability, ROI-based decision-making, real deployment components, monitoring, and Tableau dashboards for stakeholder communication. The result is both academically rigorous and aligned with real-world industry practices, making it suitable for a professional course submission and portfolio use.